

Machine Learning

Volker Roth

Department of Mathematics & Computer Science
University of Basel

Chapter 7: Support Vector Machines and Kernels

$$R[f_n] \leq R_{\text{emp}}[f_n] + \sqrt{\frac{a}{n} \left(\text{capacity}(\mathcal{H}) + \ln \frac{b}{\delta} \right)}$$

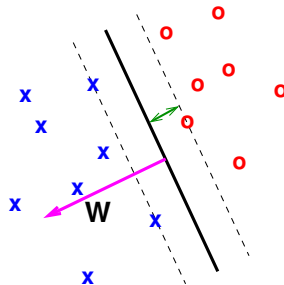
- ⇒ **minimizing $\text{capacity}(\mathcal{H})$ corresponds to maximizing the margin.**
- ⇒ **Large margin classifiers.**

- When the training examples are **linearly separable** we can maximize the margin by minimizing the regularization term

$$\|\mathbf{w}\|^2/2 = \sum_{i=1}^d w_i^2/2$$

subject to the **classification constraints**

$$y_i[\mathbf{x}_i^t \mathbf{w}] - 1 \geq 0, \quad i = 1, \dots, n.$$



- The solution is defined only on the basis of a subset of examples or **support vectors**.

SVMs: nonseparable case

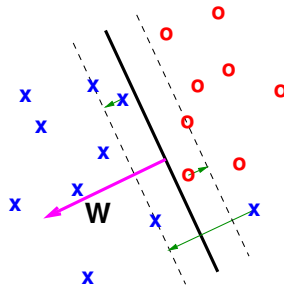
- Modify optimization problem slightly by adding a **penalty for violating the classification constraints**:

$$\text{minimize } \|\mathbf{w}\|^2/2 + C \sum_{i=1}^n \xi_i$$

subject to **relaxed constraints**

$$y_i[\mathbf{x}_i^t \mathbf{w}] - 1 + \xi_i \geq 0, \quad i = 1, \dots, n.$$

- The $\xi_i \geq 0$ are called **slack variables**.



SVMs: nonseparable case

- We can also write the SVM optimization problem more compactly as

$$C \sum_{i=1}^n \overbrace{(1 - y_i [\mathbf{x}_i^t \mathbf{w}])^+}^{\xi_i} + \|\mathbf{w}\|^2/2,$$

where $(z)^+ = z$ if $z \geq 0$ and zero otherwise.

- This is equivalent to **regularized empirical loss minimization**

$$\underbrace{\frac{1}{n} \sum_{i=1}^n (1 - y_i [\mathbf{x}_i^t \mathbf{w}])^+}_{R_{\text{emp}}} + \lambda \|\mathbf{w}\|^2,$$

where $\lambda = 1/(2nC)$ is the regularization parameter.

SVMs and LOGREG

- When viewed from the point of view of regularized empirical loss minimization, SVM and logistic regression appear quite similar:

$$\text{SVM: } \frac{1}{n} \sum_{i=1}^n (1 - y_i [\mathbf{x}_i^t \mathbf{w}])^+ + \lambda \|\mathbf{w}\|^2$$

$$\text{LOGREG: } \frac{1}{n} \sum_{i=1}^n -\log \overbrace{\sigma(y_i [\mathbf{x}_i^t \mathbf{w}])}^{P(y_i | \mathbf{x}_i, \mathbf{w})} + \lambda \|\mathbf{w}\|^2,$$

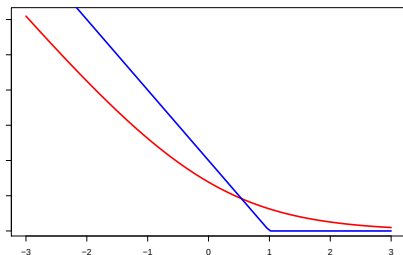
where $\sigma(z) = (1 + e^{-z})^{-1}$ is the logistic function.

SVMs and LOGREG

- The difference comes from how we penalize errors:

$$\text{Both: } \frac{1}{n} \sum_{i=1}^n \text{Loss}(\overbrace{y_i [\mathbf{x}_i^t \mathbf{w}]}^z) + \lambda \|\mathbf{w}\|^2,$$

- SVM: $\text{Loss}(z) = (1 - z)^+$
- LOGREG:
 $\text{Loss}(z) = \log(1 + \exp(-z))$



SVMs: solution, Lagrange multipliers

- Back to the separable case: how do we solve

$$\text{minimize}_{\mathbf{w}} \quad \|\mathbf{w}\|^2/2 \quad \text{s.t.} \quad y_i[\mathbf{x}_i^t \mathbf{w}] - 1 \geq 0, \quad i = 1, \dots, n.$$

- Represent the constraints as individual loss terms:

$$\sup_{\alpha_i \geq 0} \alpha_i(1 - y_i[\mathbf{x}_i^t \mathbf{w}]) = \begin{cases} 0, & \text{if } y_i[\mathbf{x}_i^t \mathbf{w}] - 1 \geq 0, \\ \infty, & \text{otherwise.} \end{cases}$$

- Rewrite the minimization problem:

$$\begin{aligned} & \text{minimize}_{\mathbf{w}} \quad \|\mathbf{w}\|^2/2 + \sum_{i=1}^n \sup_{\alpha_i \geq 0} \alpha_i(1 - y_i[\mathbf{x}_i^t \mathbf{w}]) \\ &= \text{minimize}_{\mathbf{w}} \quad \sup_{\alpha_i \geq 0} \left(\|\mathbf{w}\|^2/2 + \sum_{i=1}^n \alpha_i(1 - y_i[\mathbf{x}_i^t \mathbf{w}]) \right) \end{aligned}$$

SVMs: solution, Lagrange multipliers

- Swap maximization and minimization (technically this requires that the problem is convex and feasible $\rightsquigarrow$ **Slater's condition**):

$$\begin{aligned} & \text{minimize}_{\mathbf{w}} \left[\sup_{\alpha_i \geq 0} \left(\|\mathbf{w}\|^2/2 + \sum_{i=1}^n \alpha_i (1 - y_i [\mathbf{x}_i^t \mathbf{w}]) \right) \right] \\ &= \text{maximize}_{\alpha_i \geq 0} \left[\min_{\mathbf{w}} \underbrace{\left(\|\mathbf{w}\|^2/2 + \sum_{i=1}^n \alpha_i (1 - y_i [\mathbf{x}_i^t \mathbf{w}]) \right)}_{J(\mathbf{w}; \alpha)} \right] \end{aligned}$$

- We have to minimize $J(\mathbf{w}; \alpha)$ over parameters $\mathbf{w}$ for fixed **Lagrange multipliers** $\alpha_i \geq 0$.

Simple, because $J(\mathbf{w})$ is convex $\rightsquigarrow$ set derivative to zero
 $\rightsquigarrow$ only one stationary point $\rightsquigarrow$ global minimum.

SVMs: solution, Lagrange multipliers

- Find optimal $\mathbf{w}$ by setting the derivatives to zero:

$$\frac{\partial}{\partial \mathbf{w}} J(\mathbf{w}; \alpha) = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i = 0 \Rightarrow \hat{\mathbf{w}} = \sum_i \alpha_i y_i \mathbf{x}_i.$$

- Substitute the solution back into the objective and get (after some re-arrangements of terms):

$$\begin{aligned} & \max_{\alpha_i \geq 0} \min_{\mathbf{w}} \left(\|\mathbf{w}\|^2/2 + \sum_{i=1}^n \alpha_i (1 - y_i [\mathbf{x}_i^t \mathbf{w}]) \right) \\ &= \max_{\alpha_i \geq 0} \left(\|\hat{\mathbf{w}}\|^2/2 + \sum_{i=1}^n \alpha_i (1 - y_i [\mathbf{x}_i^t \hat{\mathbf{w}}]) \right) \\ &= \max_{\alpha_i \geq 0} \left(\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n y_i y_j \alpha_i \alpha_j \mathbf{x}_i^t \mathbf{x}_j \right) \end{aligned}$$

SVMs: summary

- Find optimal Lagrange multipliers $\hat{\alpha}_i$ by maximizing

$$\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n y_i y_j \alpha_i \alpha_j \mathbf{x}_i^t \mathbf{x}_j \quad \text{subject to } \alpha_i \geq 0.$$

- Only $\hat{\alpha}_i$'s corresponding to **support vectors** will be non-zero.
- Make **predictions** on any new example $\mathbf{x}$ according to:

$$\text{sign}(\mathbf{x}^t \hat{\mathbf{w}}) = \text{sign}(\mathbf{x}^t \sum_{i=1}^n \hat{\alpha}_i y_i \mathbf{x}_i) = \text{sign}(\sum_{i \in SV} \hat{\alpha}_i y_i \mathbf{x}^t \mathbf{x}_i).$$

- Observation: dependency on input vectors only via **dot products**.
- Later we will introduce the **kernel trick** for efficiently computing these dot products in implicitly defined feature spaces.

SVMs: formal derivation

- **Convex optimization problem:** an optimization problem

$$\text{minimize} \quad f(\mathbf{x}) \quad (1)$$

$$\text{subject to} \quad g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \quad (2)$$

is convex if the functions $f, g_1 \dots g_m : \mathbb{R}^n \rightarrow \mathbb{R}$ are convex.

- The **Lagrangian function** for the problem is

$$\mathcal{L}(\mathbf{x}, \lambda_0, \dots, \lambda_m) = \lambda_0 f(\mathbf{x}) + \lambda_1 g_1(\mathbf{x}) + \dots + \lambda_m g_m(\mathbf{x}).$$

- **Karush-Kuhn-Tucker (KKT) conditions:** For each point $\hat{\mathbf{x}}$ that minimizes f , there exist real numbers $\lambda_0, \dots, \lambda_m$, called **Lagrange multipliers**, that simultaneously satisfy:

- 1 $\hat{\mathbf{x}}$ minimizes $\mathcal{L}(\mathbf{x}, \lambda_0, \lambda_1, \dots, \lambda_m)$,
- 2 $\lambda_0 \geq 0, \lambda_1 \geq 0, \dots, \lambda_m \geq 0$, with at least one $\lambda_k > 0$,
- 3 Complementary slackness: $g_i(\hat{\mathbf{x}}) < 0 \Rightarrow \lambda_i = 0, 1 \leq i \leq m$.

SVMs: formal derivation

- **Slater's condition:** If there exists a **strictly feasible point** z satisfying $g_1(z) < 0, \dots, g_m(z) < 0$, then one can set $\lambda_0 = 1$.
- Assume that Slater's condition holds. Minimizing the supremum $\mathcal{L}^*(\mathbf{x}) = \sup_{\lambda \geq 0} \mathcal{L}(\mathbf{x}, \lambda)$, is the **primal problem P**:

$$\hat{\mathbf{x}} = \operatorname{argmin}_{\mathbf{x}} \mathcal{L}^*(\mathbf{x}).$$

Note that

$$\mathcal{L}^*(\mathbf{x}) = \sup_{\lambda \geq 0} \left(f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) \right) = \begin{cases} f(\mathbf{x}) & , \text{ if } g_i(\mathbf{x}) \leq 0 \forall i \\ \infty & , \text{ else.} \end{cases}$$

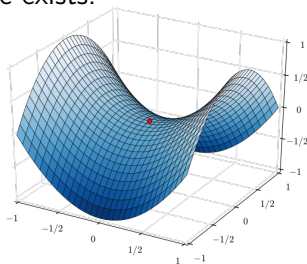
$\leadsto$ **Minimizing $\mathcal{L}^*(\mathbf{x})$ is equivalent to minimizing $f(\mathbf{x})$.**

- The maximizer of the **dual problem D** is

$$\hat{\lambda} = \operatorname{argmax}_{\lambda} \mathcal{L}_*(\lambda), \text{ where } \mathcal{L}_*(\lambda) = \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda).$$

SVMs: formal derivation

- The non-negative number $\min(P) - \max(D)$ is the **duality gap**.
- **Convexity and Slater's condition imply strong duality:**
 - 1 The optimal solution $(\hat{x}, \hat{\lambda})$ is a saddle point of $\mathcal{L}(x, \lambda)$
 - 2 The **duality gap is zero**.
- Discussion: For any real function $f(a, b)$
 $\min_a[\max_b f(a, b)] \geq \max_b[\min_a f(a, b)]$.
Equality $\rightsquigarrow$ saddle value exists.



By Nicoguaro - Own work, CC BY 3.0, <https://commons.wikimedia.org/w/index.php?curid=20570051>

Kernel functions

- A **kernel function** is a real-valued function of two arguments, $k(\mathbf{x}, \mathbf{x}') \in \mathbb{R}$, for $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$.
- Typically the function is **symmetric**, and sometimes non-negative.
- In the latter case, it might be interpreted as a measure of similarity.
- Example: **isotropic Gaussian kernel**:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

Here, σ^2 is the bandwidth. This is an example of a **radial basis function (RBF) kernel** (only a function of $\|\mathbf{x} - \mathbf{x}'\|^2$).

Mercer kernels

- A symmetric kernel is a **Mercer kernel**, iff the Gram matrix

$$K = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \dots & k(\mathbf{x}_1, \mathbf{x}_n) \\ & \ddots & \\ k(\mathbf{x}_n, \mathbf{x}_1) & \dots & k(\mathbf{x}_n, \mathbf{x}_n) \end{pmatrix}$$

is **positive semidefinite** for any set of inputs $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$.

- Mercer's theorem:** Eigenvector decomposition

$$K = V\Lambda V^t = (V\Lambda^{1/2})(V\Lambda^{1/2})^t =: \Phi\Phi^t.$$

Eigenvectors: columns of V . Eigenvalues: entries of diagonal matrix $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$. Note that $\lambda_i \in \mathbb{R}$ and $\lambda_i \geq 0$.

Define $\phi(\mathbf{x}_i)^t = i$ -th row of $\Phi = V_{[i, \bullet]}\Lambda^{1/2}$

$$\rightsquigarrow k(\mathbf{x}_i, \mathbf{x}_{i'}) = \phi(\mathbf{x}_i)^t \phi(\mathbf{x}_{i'}).$$

- Entries of K : **inner product of some feature vectors**, implicitly defined by eigenvectors V .

Mercer kernels

- If the kernel is **Mercer**, then there exists $\phi : \mathbf{x} \rightarrow \mathbb{R}^d$ such that

$$k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^t \phi(\mathbf{x}'),$$

where ϕ depends on the eigenfunctions of k (d might be infinite).

- Example: **Polynomial kernel**

$$k(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^t \mathbf{x}')^m.$$

Corresponding feature vector contains terms up to degree m .

Example: $m = 2$, $\mathbf{x} \in \mathbb{R}^2$:

$$(1 + \mathbf{x}^t \mathbf{x}')^2 = 1 + 2x_1x_1' + 2x_2x_2' + (x_1x_1')^2 + (x_2x_2')^2 + 2x_1x_1'x_2x_2'.$$

Thus,

$$\phi(\mathbf{x}) = [1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2]^t.$$

Equivalent to working in a 6-dim feature space.

- **Gaussian kernel:** feature map lives in an infinite dimensional space.

Kernels for documents

- In document classification or retrieval, we want to compare two documents, $\mathbf{x}_i$ and $\mathbf{x}_{i'}$.
- **Bag of words** representation:
 x_{ij} is the number of times word j occurs in document i .

- One possible choice: **Cosine similarity:**

$$k(\mathbf{x}_i, \mathbf{x}_{i'}) = \frac{\mathbf{x}_i^t \mathbf{x}_{i'}}{\|\mathbf{x}_i\| \|\mathbf{x}_{i'}\|} =: \phi(\mathbf{x}_i)^t \phi(\mathbf{x}_{i'}).$$

- Problems:
 - ▶ Popular words (like “the” or “and”) are not discriminative
 $\leadsto$ remove these **stop words**.
 - ▶ Bias: once a word is used in a document,
 it is very likely to be **used again**.
- Solution: Replace word counts with “normalized” representation.

Kernels for documents

- TF-IDF “term frequency inverse document frequency”:

Term frequency is log-transform of the count:

$$\text{tf}(x_{ij}) = \log(1 + x_{ij})$$

Inverse document frequency:

$$\text{idf}(j) = \log \frac{\#(\text{documents})}{\#(\text{documents containing term } j)} = \log \frac{1}{\hat{p}_j}.$$

↪ Shannon information content:

idf is a measure of how much information a word provides

- Combine with tf ↪ counts weighted by information content:

$$\text{tf-idf}(\mathbf{x}_i) = [\text{tf}(\mathbf{x}_{ij}) \cdot \text{idf}(j)]_{j=1}^V, \text{ where } V = \text{size of vocabulary.}$$

- We then use this inside the **cosine similarity measure**.

With $\phi(\mathbf{x}) = \text{tf-idf}(\mathbf{x})$:

$$k(\mathbf{x}_i, \mathbf{x}_{i'}) = \frac{\phi(\mathbf{x}_i)^t \phi(\mathbf{x}_{i'})}{\|\phi(\mathbf{x}_i)\| \|\phi(\mathbf{x}_{i'})\|}.$$

String kernels

- Real power of kernels arises for **structured input objects**.
- Consider two strings x , and x' of lengths d, d' , over alphabet $\mathcal{A}$.
Idea: define similarity as the **number of common substrings**.
- If s is a substring of $x \rightsquigarrow \phi_s(x) = \text{number of times } s \text{ appears in } x$.
- **String kernel**

$$k(x, x') = \sum_{s \in \mathcal{A}^*} w_s \phi_s(x) \phi_s(x'),$$

where $w_s \geq 0$ and $\mathcal{A}^* = \text{set of all strings (any length) from } \mathcal{A}$.

- One can show: Mercer kernel, can be computed in $O(|x| + |x'|)$ time using suffix trees (Shawe-Taylor and Cristianini, 2004).
- Special case: $w_s = 0$ for $|s| > 1$: **bag-of-characters kernel**:
 $\phi(x)$ is the number of times each character in $\mathcal{A}$ occurs in x .

The kernel trick

- Idea: modify algorithm so that it **replaces all inner products $\mathbf{x}^t \mathbf{x}'$** with a call to the **kernel function $k(\mathbf{x}, \mathbf{x}')$** .
- **Kernelized ridge regression:** $\hat{\mathbf{w}} = (X^t X + \lambda I)^{-1} X^t \mathbf{y}$.

Matrix inversion lemma:

$$(I + UV)^{-1} U = U(I + VU)^{-1}$$

Define new variables α_i :

$$\begin{aligned}\hat{\mathbf{w}} &= (X^t X + \lambda I)^{-1} X^t \mathbf{y} \\ &= X^t \underbrace{(X X^t + \lambda I)^{-1}}_{\hat{\alpha}} \mathbf{y} = \sum_{i=1}^n \hat{\alpha}_i \mathbf{x}_i.\end{aligned}$$

$\leadsto$ **solution is linear sum of the n training vectors.**

The kernel trick

- Use this and the kernel trick to make **predictions for $\mathbf{x}$:**

$$\hat{f}(\mathbf{x}) = \hat{\mathbf{w}}^t \mathbf{x} = \sum_{i=1}^n \hat{\alpha}_i \mathbf{x}_i^t \mathbf{x} = \sum_{i=1}^n \hat{\alpha}_i k(\mathbf{x}_i, \mathbf{x}).$$

- Same for SVMs:

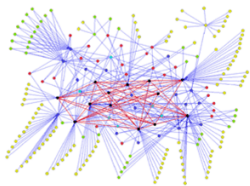
$$\hat{\mathbf{w}}^t \mathbf{x} = \sum_{i \in SV} \hat{\alpha}_i y_i \mathbf{x}_i^t \mathbf{x} = \sum_{i \in SV} \hat{\alpha}'_i k(\mathbf{x}_i, \mathbf{x})$$

- ...and for most other classical algorithms in ML!

Some applications in bioinformatics

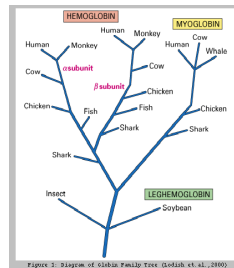
- Bioinformatics: often **non-vectorial** data-types:

- ▶ interaction graphs



- ▶ phylogenetic trees

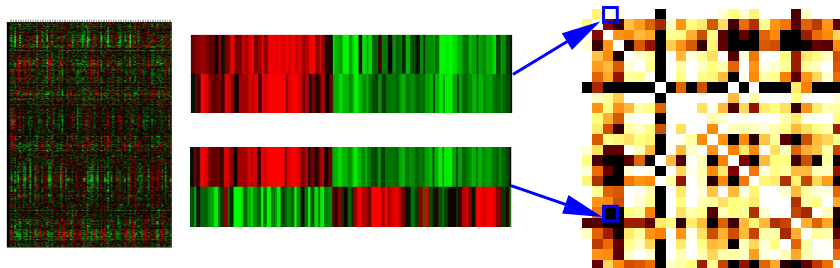
- ▶ strings **GSAQVKGHGKKVADALTNAVAHV**



- **Data fusion:** convert data of each type into kernel matrix
 - ⇒ fuse kernel matrices
 - ⇒ “common language” for heterogeneous data.

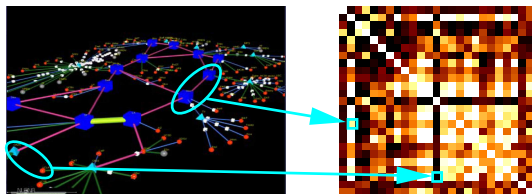
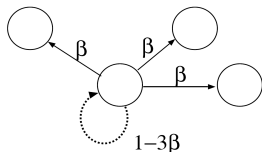
RBF kernels from expression data

- **Measurements** (for each gene): vector of expression values under different experimental conditions
- “classical” RBF kernel $k(x_1, x_2) = \exp(-\sigma \|x_1 - x_2\|^2)$



Diffusion kernels from interaction-graphs

- A : Adjacency matrix, D : node degrees, $L = D - A$.
- $K := \frac{1}{Z(\beta)} \exp(-\beta L)$ with transition probabilities β .
- **Physical interpretation (random walk):**
randomly choose next node among neighbors.
- Self-transition occurs with prob. $1 - d_i\beta$



- K_{ij} : prob. for walk from i to j .

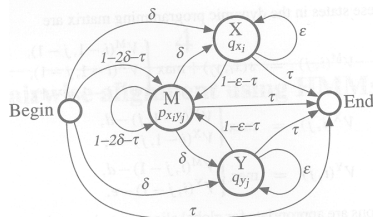
(Kondor and Lafferty, 2002)

Alignment kernels from sequences

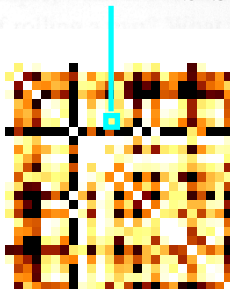
Alignment with **Pair HMMs**

↪ Mercer kernel (Watkins, 2000).

Image source: Durbin, Eddy, Krogh, Mitchison. Biological Sequence Alignment. Cambridge.



```
HBA_HUMAN  GSAQVKGHGKKVADALTNAVAHV---D--DMPNALSALSDDLHAHKL
              ++  +++++H+  KV    +  +A  ++                +L+  L++++H+  K
LGB2_LUPLU  NNPELQAHAGKVFKLVYEAAIQLQVTGVVVTDATLKNLGSVHVS KG
```



Combination of heterogeneous data

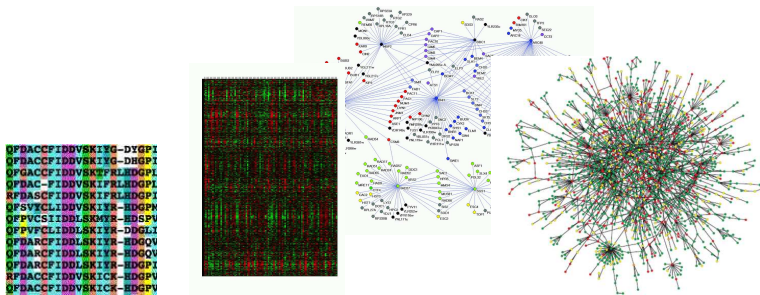
Adding kernels $\Rightarrow$ new kernel:

$$k_1(x, y) = \phi_1(x) \cdot \phi_1(y),$$

$$k_2(x, y) = \phi_2(x) \cdot \phi_2(y)$$

$$\Rightarrow k' = k_1 + k_2 = \begin{pmatrix} \phi_1(x) \\ \phi_2(x) \end{pmatrix} \cdot \begin{pmatrix} \phi_1(y) \\ \phi_2(y) \end{pmatrix}$$

Fusion & relevance determination: kernel-combinations



$$\boxed{K} = c_1 \boxed{K_1} + c_2 \boxed{K_2} + c_3 \boxed{K_3} + c_4 \boxed{K_4}$$